

Face Verification Hardware-Aware Profiling Report

Group Name: ML Verifier

Members: Moataz Osama Saadeldin Abdelaziz, Ankan Roy

Overview

This report presents a hardware-aware profiling analysis of the final face verification system. The goal is to characterize the runtime behavior of the inference pipeline under a CPU-only environment and to identify which stages dominate latency. The profiling focuses on:

- Single-pair inference latency
- Batch-size sensitivity across repeated pair-level inference requests
- Per-stage latency breakdown: preprocessing, embedding generation and scoring
- End-to-end inference latency
- Throughput in pairs per second
- p95 total latency

CPU Environment

All experiments were conducted using a CPU baseline. No GPU measurements were included, as the focus of this report is to establish a consistent CPU reference point.

- Processor: 2.3 GHz 8-Core Intel Core i9
- Execution Mode: CPU-only, no GPU acceleration
- Framework: Python-based inference pipeline using embedding-based verification
- Model: InsightFace buffalo_s
- Score type: cosine_with_embeddings
- Operating threshold: 0.29
- Profiling script: scripts/run_profiling.py
- Profiling output file: reports/profiling_cpu_summary.csv

Single-Pair and Batch-Level Inference

The system was profiled using high-resolution timing with `time.perf_counter` to measure inference latency under a CPU-only setup. The pipeline was split into three stages: preprocessing, embedding generation, and similarity scoring. Experiments were conducted for both single-pair inference and batch inference using a fixed set of images to ensure consistency across measurements.

The profiling script reused the final inference path from `src/run_inference_cli.py`, so the profiling results are aligned with the same system used by the CLI.

Batch size means the number of image-pair inference requests processed in one profiling group. Each request compares two images. Each batch size was repeated 3 times.

Latency was measured independently for preprocessing, embedding generation, and similarity scoring to understand the contribution of each stage to overall runtime. Preprocessing prepares the image format before embedding extraction, embedding generation involves face detection and feature extraction using the model, and scoring consists of cosine similarity computation using the predefined threshold.

Single-pair inference measures the time required to process one image pair end-to-end, while batch-size sensitivity evaluates performance when multiple image-pair inference requests are processed in one profiling group. This comparison highlights how the system behaves under different workloads.

CPU Profiling Results

Batch Size	Repeats	Total Requests	Preprocessing Mean (ms)	Embedding Mean (ms)	Scoring Mean (ms)	Total Mean (ms)	Total P95 (ms)	Throughput (pairs/sec)
1	3	3	0.012	293.585	0.021	293.618	135.736	0.738
2	3	6	0.009	112.017	0.013	112.039	130.111	8.777
4	3	12	0.009	89.653	0.013	89.675	120.193	10.931
8	3	24	0.008	72.065	0.013	72.086	116.722	13.473

The results show that embedding generation is the dominant contributor to total latency, significantly outweighing preprocessing and scoring costs. Preprocessing introduces minimal overhead, while similarity scoring remains computationally negligible.

The batch-size results show that throughput improves as the batch size increases. Throughput increases from 0.738 pairs per second at batch size 1 to 13.473 pairs per second at batch size 8.

The total mean latency also decreases across larger batch sizes, from 293.618 ms at batch size 1 to 72.086 ms at batch size 8. This supports the conclusion that the main runtime cost is embedding generation, while preprocessing and scoring are minimal.

The p95 latency values provide a simple estimate of slower inference runs. Because each batch size was repeated only 3 times, the p95 values should be treated as a small local profiling estimate rather than a large-scale benchmark.

Limitations

The profiling results are based on a single CPU configuration and may not generalize to other hardware environments or deployment settings. Since GPU profiling was not included, this report does not measure possible GPU speedups.

The profiling uses a small set of repeated pair-level inference runs. It is sufficient for a local CPU baseline and batch-size sensitivity analysis, but it should not be interpreted as a large-scale production benchmark.